

Synthetic Python -> Jac data and a small finetune probe.

Teaching a model to write Jac

Ayush Madhav Kumar



JASECI

No Real Jac Corpus Exists

Models trained on Python have weak priors on Jac's data-spatial idioms: walkers, nodes and edges instead of functions and classes.

Stock 30B and 26B models of Gemma4 and Qwen3-Coder produce 0% runnable Jac cause they have no previous training of the same !

```
# a graph walk written as a loop
def traverse(graph, start):
    stack = [start]
    seen = set()
    while stack:
        n = stack.pop()
        seen.add(n)
        stack += graph[n]
    return seen
```

```
# the graph walks itself
walker Traverse {
    can step with Item entry {
        here.seen = True;
        visit [-->];
    }
}
node Item { has seen: bool = False; }
```

Three Anchors for 100% synthetic data

1. Grammar as the distribution anchor : The Jac grammar defines the space of valid programs. Every sample is shaped to fit it.
2. Compiler and tests as a “free oracle: Unlimited , zero cost labelling. Reject anything that doesn’t compile, run or pass behavioral tests.
3. Python as the proxy distribution : Translate validated python into jac to borrow a large, correct data source.

This is the multi PLT method.

The quality gate is jac run : it compiles, runs and the output must match. Never the type checker which over rejects runnable code.

Data Tiers pt1

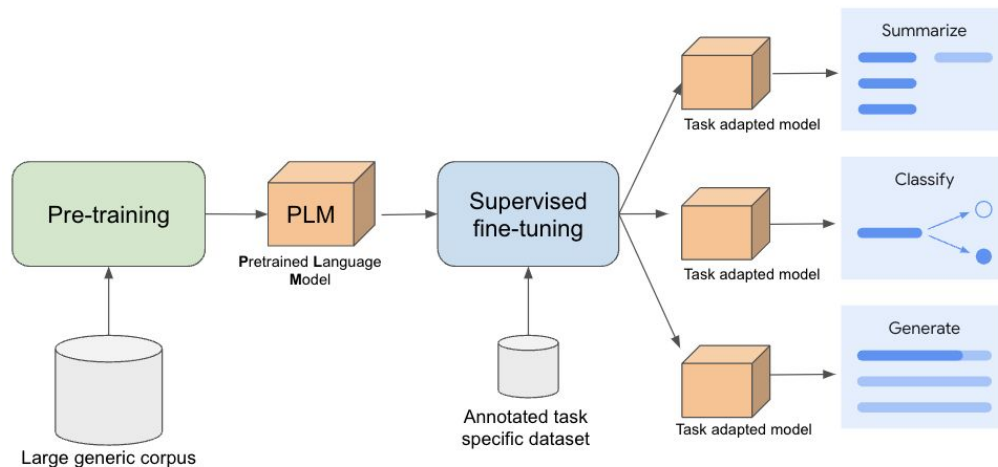
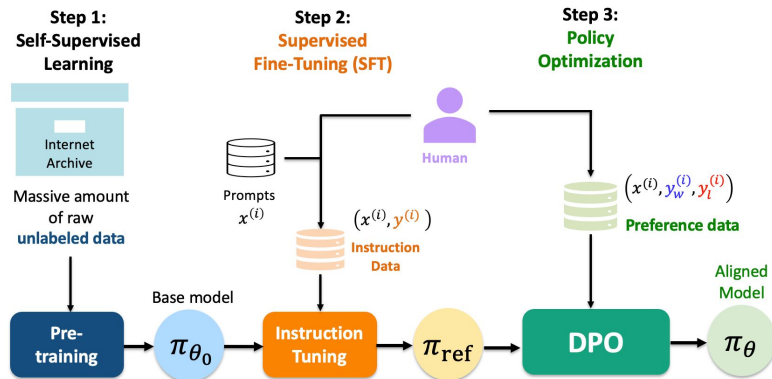
1. Idiomatic Core [147] : Jac , by “hand” : Walkers, nodes and edges, written agentically with claude code and jac mcp. 31 are graph shaped.
2. Transpile Volume [1500] : Python shaped : Auto transpiled python, behaviorally gated, correct but not idiomatic
3. DPO pairs [147] : De-Python-Ification: chosen = idiomatic , rejected = transpile . teaches the model whats right whats wrong.

Data tiers pt2

What is SFT and DPO

- SFT: Supervised Fine Tuning
- DPO: Direct Preference Optimization

Direct Preference Optimization (DPO)



Function vs Graph Tasks

Graphs and stats

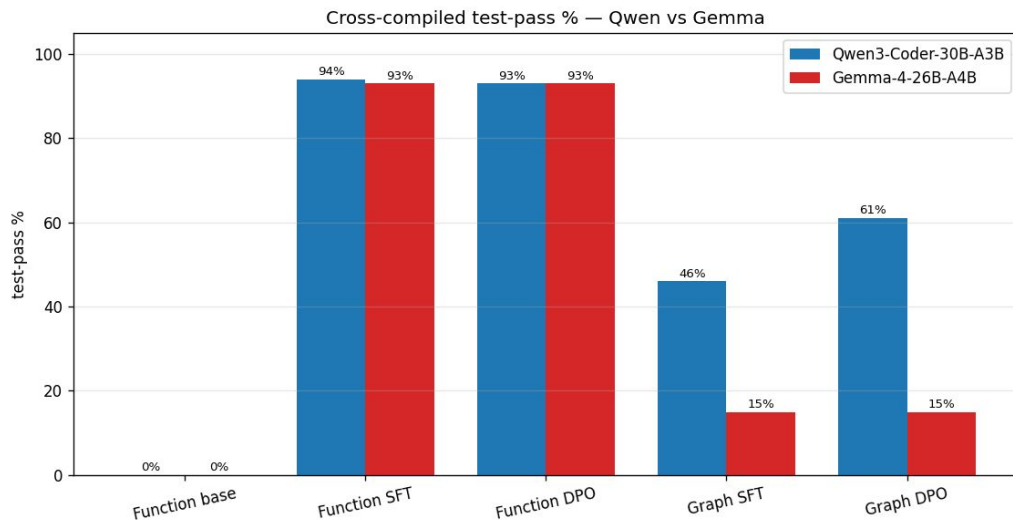
Base Eval : from cold start

Quantized and ran normal , q4, q8

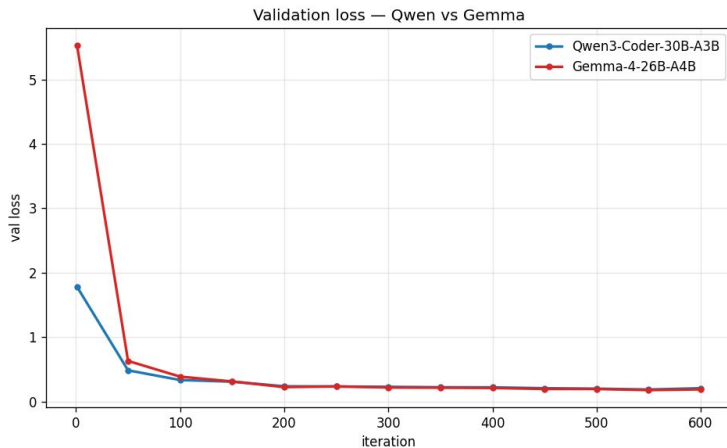
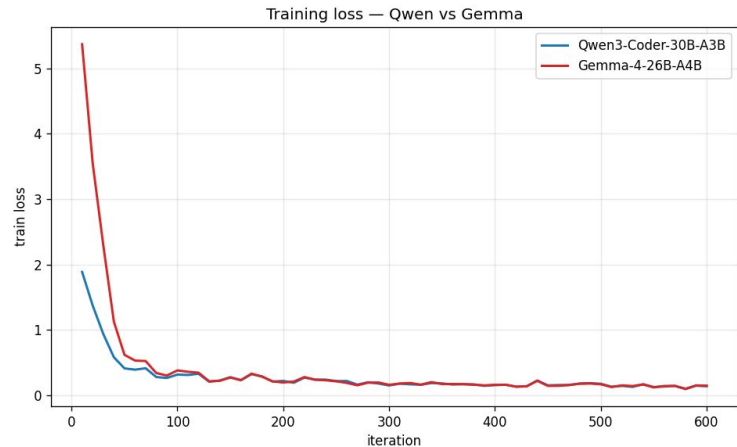
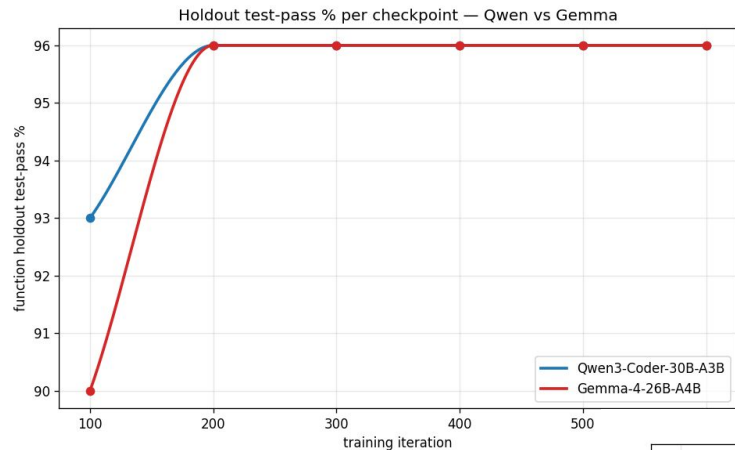
Gemma 4 and Qwen3 Coder

Both models essentially learned to transpile. Outputs stayed python shaped and DPO did not move them.

Base was 0 because both models had no idea about jac before this.



Graphs and stats pt2



Very very similar
results for the
validation and
training loss and
also for the
learning curves !

Graph tier: Where idiom has room to move

On graph shaped tasks, where you reach for nodes, edges and walkers instead of a dict and a stack, idiomatic jac diverges from python.

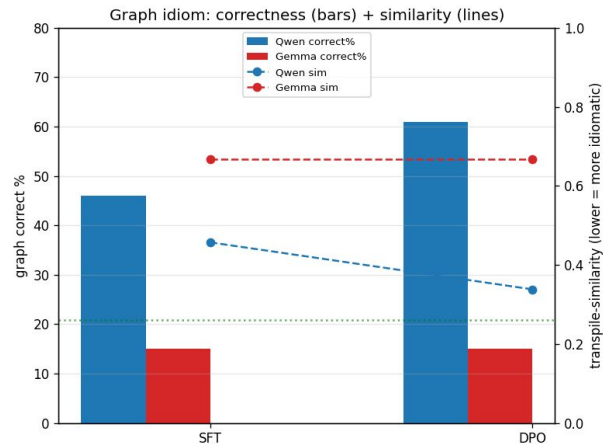
Here is where the DPO comes into affect.

SFT teaches the model how to run.

DPO pushes it the rest of the way, from runs to idiomatic.

Qwen reaches 100% idiomatic.

Gemma learns graph idiom for weaker (15%, 0 graph constructs) so idiom acquisition depends entirely on what model we use.



idiomatic ref 0.26

What We Proved

1. Synthetic Compiler validated data teaches a model correct jac with an accuracy of 94%
2. Idiot headroom plus DPO teaches a model (Qwen3Coder) idiomatic jac 100% of correct outputs idiomatic , where it was 83% at SFT.
3. Therefore, there we have it , a very basic jac coding agent , essentially what claude code is but for jac instead of python and other popular programming languages.

Next Up

Mine a single codebase into an rl loop, see how far the resulting codebase is from what you asked it to do
Is it ridiculous to use one codebase to get models to be smart
What is the kind of eval that you would use for such an evaluation

I would like to answer these questions over the next week as well as build a training dashboard that shows the training of a model as it happens in real time.